

DE1-SoC Neural Navigation Robot

Technical Setup Report

FPGA-Accelerated Obstacle Avoidance Using a Trained Neural Network

Platform: Altera DE1-SoC (Cyclone V 5CSEMA5F31C6)

Architecture: $3 \rightarrow 8$ (ReLU) $\rightarrow 3 \rightarrow \text{argmax}$

Inference Latency: $\sim 1\mu\text{s}$ at 50 MHz

Fully Portable: Battery-powered operation

Author: Mouhamad A. Al-Amine

Date: May 2026

Table of Contents

1. System Overview	3
2. Hardware Requirements	3
2.1 Component List.....	3
2.2 DE1-SoC Board Resources Used.....	4
3. Wiring and Connections	4
3.1 Ultrasonic Sensors (HC-SR04).....	4
3.2 UART Communication (Arduino ↔ FPGA).....	4
3.3 Motor Drivers (Dual L298N).....	5
3.4 Power Distribution.....	6
4. System Architecture	6
4.1 Data Flow	6
4.2 Communication Protocol.....	6
4.3 Neural Network Architecture.....	7
4.4 FPGA Board Controls	7
5. Software and Firmware	8
5.1 FPGA Verilog Modules	8
5.2 Arduino Sketches.....	8
5.3 Training Pipeline	8
6. Neural Network Training.....	9
6.1 Data Collection	9
6.2 Labeling Rules	9
6.3 Data Augmentation	9
6.4 Quantization.....	9
7. Deployment Instructions.....	10
7.1 FPGA Programming	10
8. Performance Summary.....	10
9. Attached Code Files	11

1. System Overview

This project implements a fully autonomous indoor obstacle-avoidance robot powered by a neural network deployed on FPGA fabric. The system uses three fixed ultrasonic distance sensors to perceive the environment, an Arduino Uno as a sensor/motor interface, and the DE1-SoC FPGA board as the AI decision-making brain.

The neural network runs entirely in hardware (no CPU, no software), achieving ~1 microsecond inference latency. It classifies each sensor reading into one of three actions: FORWARD (class 0), STOP (class 1), or TURN (class 2). The Arduino executes the FPGA's decision without override, making the FPGA the sole decision-maker.

The entire system operates untethered on battery power: a 7.4V LiPo powers the motors and Arduino, while a second LiPo with a DC boost converter powers the DE1-SoC board (which requires 12V). This allows fully portable outdoor and hallway demonstrations with no cables attached.

Key achievements: zero collisions during live demonstration, 100ms control loop, and only 222 ALMs + 2 DSP blocks of FPGA resources used.

2. Hardware Requirements

2.1 Component List

Component	Qty	Notes
DE1-SoC Board (Cyclone V)	1	5CSEMA5F31C6, 85K LEs, 50 MHz clock
Arduino Uno R3	1	Sensor interface + motor driver
HC-SR04 Ultrasonic Sensor	3	Left (45°), Center (0°), Right (45°)
L298N Motor Driver	2	Dual H-bridge, drives 4 DC motors
4-Wheel Robot Chassis	1	With 4 DC gear motors
Bi-directional Level Converter	1	3.3V (FPGA) ↔ 5V (Arduino) UART
7.4V 2S LiPo Battery (motors + FPGA)	2	Powers motors via L298N + Arduino via 5V reg
DC Boost Converter	1	Boosts to 12V to power DE1-SoC portably
Jumper Wires (M-M, M-F)	~30	For all connections
Breadboard	1	Level converter + sensor distribution

2.2 DE1-SoC Board Resources Used

Resource	Used	Available
ALMs (Adaptive Logic Modules)	222	32,070
DSP Blocks	2	87
GPIO Pins	2 (UART)	36 on GPIO_0
Clock	50 MHz	CLOCK_50
7-Segment Displays	1 (HEX0)	6 total
LEDs	4 (LEDR[3:0])	10 total
Switches	1 (SW[0])	10 total
Keys (Push Buttons)	2 (KEY[0:1])	4 total

3. Wiring and Connections

3.1 Ultrasonic Sensors (HC-SR04)

All three sensors share a single TRIG pin (A0) since they are triggered sequentially with 30ms delays between reads. Each sensor has its own ECHO pin for independent distance measurement. Sensors are mounted at the front of the chassis at approximately 45° left, 0° center, and 45° right.

Sensor	Arduino Pin	Signal	Mounting Angle
All 3 sensors	A0	TRIG (shared)	—
Left HC-SR04	A1	ECHO	~45° left of center
Center HC-SR04	A2	ECHO	0° (straight ahead)
Right HC-SR04	A3	ECHO	~45° right of center

3.2 UART Communication (Arduino ↔ FPGA)

The Arduino and FPGA communicate via UART at 9600 baud (8N1). A bi-directional level converter is REQUIRED because the FPGA GPIO operates at 3.3V while the Arduino operates at 5V. Directly connecting 5V signals to the FPGA will damage it.

Arduino Side	Level Conv. (5V)	Level Conv. (3.3V)	FPGA Side
A4 (Soft Serial TX)	HV1	LV1	GPIO_0[0] (PIN_AC18)
A5 (Soft Serial RX)	HV2	LV2	GPIO_0[1] (PIN_Y17)
GND	GND (HV)	GND (LV)	GND (GPIO header)
5V	HV (5V ref)	LV (3.3V ref)	3.3V (GPIO header)

3.3 Motor Drivers (Dual L298N)

Two L298N boards drive the four chassis motors. Each L298N controls one side (left pair or right pair). PWM speed control via ENA/ENB pins; direction via IN1-IN4 logic pins.

Arduino Pin	L298N Pin	Function	Side
D5 (PWM)	L1 ENA	Left-Front speed	Left
D6 (PWM)	L1 ENB	Left-Rear speed	Left
D4	L1 IN1	Left-Front dir A	Left
D7	L1 IN2	Left-Front dir B	Left
D8	L1 IN3	Left-Rear dir A	Left
D12	L1 IN4	Left-Rear dir B	Left
D11 (PWM)	L2 ENA	Right-Front speed	Right
D3 (PWM)	L2 ENB	Right-Rear speed	Right
D9	L2 IN1	Right-Front dir A	Right
D13	L2 IN2	Right-Front dir B	Right
D10	L2 IN3	Right-Rear dir A	Right
D2	L2 IN4	Right-Rear dir B	Right

3.4 Power Distribution

The system is fully portable with no tethered power cables. Two separate battery systems ensure clean power isolation between the noisy motor circuits and the sensitive FPGA logic:

Power Source	Supplies	Details
7.4V 2S LiPo #1	L298N motor drivers (VS input)	7.4V direct; drives all 4 motors
L298N 5V regulator	Arduino (VIN) + HC-SR04 sensors	5V from L298N onboard regulator
LiPo #2 + Boost Converter	DE1-SoC FPGA board	LiPo boosted to 12V DC via boost converter, connects to DE1-SoC barrel jack

Important: The DE1-SoC requires 12V DC input. A DC-DC boost converter steps up the LiPo voltage (7.4–11.1V depending on cell count) to a stable 12V. This eliminates the need for a wall adapter during demonstrations, allowing the robot to drive freely in hallways and open spaces. All grounds (motor LiPo, boost converter, Arduino, FPGA GPIO) must share a common ground connection.

4. System Architecture

4.1 Data Flow

The system operates in a continuous 100ms control loop:

1. Arduino reads 3 ultrasonic distances sequentially (30ms each, ~90ms total).
2. Arduino maps each distance (0–400 cm) to a byte (0–255) and sends 3 bytes to FPGA via UART.
3. FPGA receives 3 bytes, scales to Q6.10 fixed-point [0, 1] range, triggers neural network.
4. Neural network computes forward pass in ~50 clock cycles (~1 μ s), outputs class (0/1/2).
5. FPGA sends 1-byte class back to Arduino via UART.
6. Arduino executes the action: FORWARD, STOP, or TURN (direction chosen by comparing left vs. right distance).

4.2 Communication Protocol

UART: 9600 baud, 8 data bits, no parity, 1 stop bit (8N1).

Direction	Payload	Description
Arduino → FPGA	3 bytes: [left, center, right]	Distance mapped 0-400cm → 0-255
FPGA → Arduino	1 byte: [0x00–0x02]	Class: 0=FWD, 1=STOP, 2=TURN

4.3 Neural Network Architecture

A compact 2-layer fully-connected network, chosen to fit within FPGA constraints:

Layer	Shape	Activation	Parameters
Input	3 neurons	—	—
Hidden	8 neurons	ReLU	24 weights + 8 biases
Output	3 neurons	Argmax	24 weights + 3 biases
Total	—	—	59 parameters

Arithmetic: Signed 16-bit Q6.10 fixed-point. Multiplication accumulates into 32-bit, then truncates back to 16-bit via acc[25:10]. A single multiplier is time-shared across all neurons to save DSP blocks.

4.4 FPGA Board Controls

Control	Function	State
SW[0]	Master enable	UP = active, DOWN = silent
KEY[0]	Reset (active-low)	Press to reset all state
KEY[1]	Emergency kill switch	Press = force STOP class
LEDR[1:0]	Current class display	00=FWD, 01=STOP, 10=TURN
LEDR[2]	Active indicator	ON when SW[0] is enabled
LEDR[3]	Kill switch indicator	ON when KEY[1] pressed
HEX0	7-segment class display	F=Forward, S=Stop, t=Turn

5. Software and Firmware

5.1 FPGA Verilog Modules

The Quartus project contains four Verilog source files:

File	Description
robot_neural.v	Top-level: byte collector, input scaling (byte×4 → Q6.10), UART instantiation, kill switch logic, LED/7-seg status display
neural_net.v	Sequential NN: state machine cycles Layer1 (3×8 + ReLU) → Layer2 (8×3) → Argmax. All 59 weights hardcoded as Q6.10 hex constants.
uart_rx.v	UART receiver: 9600 baud, 50 MHz, 8N1, 2-stage metastability synchronizer
uart_tx.v	UART transmitter: 9600 baud, 50 MHz, 8N1
pin_assignments.tcl	Pin location assignments for all used FPGA I/O (run in Quartus TCL console)

5.2 Arduino Sketches

File	Description
de1soc_phase5_3sensor.ino	Main robot control: reads 3 sensors, sends to FPGA, executes returned class. Includes 5s stuck recovery. MOTOR_SPEED=120.
de1soc_data_collect_v2.ino	Data collection variant: same loop but logs CSV over Serial Monitor for training data capture. MOTOR_SPEED=100.

5.3 Training Pipeline

File	Description
fpga_nn_v5.ipynb	Complete Jupyter notebook: data loading, rule-based labeling, L↔R mirroring, PyTorch training (L2 decay 0.0005), Q6.10 quantization, bit-accurate FPGA simulation, scenario testing, Verilog weight export

6. Neural Network Training

6.1 Data Collection

Training data was collected by driving the robot in a real indoor environment (hallways, rooms with furniture) using the data collection sketch. The robot drives using the current FPGA model while logging all sensor readings and FPGA decisions to Serial Monitor as CSV. Total raw samples: 1,698 rows. After removing invalid readings (class = -1) and deduplicating stuck/repetitive data (threshold: any sensor must change by >3 cm), 670 clean samples remained.

6.2 Labeling Rules

Labels are assigned algorithmically based on sensor distances (in cm) using safety-first rules:

Class	Condition	Rationale
STOP (1)	left < 15 AND center < 15 AND right < 15	Truly cornered, no safe exit
TURN (2)	left < 25 OR right < 25 OR center < 40	Obstacle on any side
FORWARD (0)	center ≥ 40 AND left ≥ 25 AND right ≥ 25	All directions clear

Priority: STOP checked first (most restrictive), then TURN, then FORWARD. This ensures the robot never drives forward when any side is dangerously close.

6.3 Data Augmentation

Left↔Right mirroring: every sample [L, C, R] generates a mirror [R, C, L] with the same label. This eliminates directional bias. Synthetic corner cases are also added for underrepresented STOP scenarios to ensure balanced training across all three classes.

6.4 Quantization

Weights are quantized to signed 16-bit Q6.10 fixed-point: multiply float weight by 1024, round to integer, clamp to $[-32768, +32767]$. L2 weight decay (0.0005) during training keeps weights small to prevent overflow. FPGA simulation uses bit-accurate integer arithmetic matching the Verilog state machine exactly.

7. Deployment Instructions

7.1 FPGA Programming

1. Open Quartus Prime Lite 25.1 and load the project file robot_neural.qpf.
2. Verify all four .v files are listed. Run full Compilation (Analysis & Synthesis → Fitter → Assembler).
3. Connect DE1-SoC via USB-Blaster. Open Programmer, select the .sof file, click Start.
4. Verify HEX0 shows 'F' (Forward) at idle and LEDR[2] is OFF (SW[0] still down).
5. 7.2 Arduino Upload
6. Open de1soc_phase5_3sensor.ino in Arduino IDE.
7. Select Board: Arduino Uno, Port: the correct COM port.
8. Upload. Open Serial Monitor at 9600 baud to verify motor self-test passes.
9. 7.3 Running the Robot (Portable Mode)
10. Connect UART wires: Arduino A4/A5 ↔ Level Converter ↔ FPGA GPIO_0[0:1].
11. Connect motor LiPo to L298N VS inputs. Arduino receives 5V from L298N regulator.
12. Connect FPGA LiPo through boost converter (set to 12V) to DE1-SoC barrel jack.
13. Flip SW[0] UP on the DE1-SoC to enable inference. Robot begins navigating autonomously.
14. Emergency stop: press KEY[1] on FPGA (forces STOP) or flip SW[0] DOWN (disables all response).

8. Performance Summary

Metric	Value
Inference latency	~1 μ s (50 clock cycles at 50 MHz)
Control loop period	~100 ms (sensor-limited)
FPGA resource usage	222 ALMs, 2 DSP blocks (<1% of chip)
Collisions during live demo	0 (zero)
FORWARD-when-should-TURN errors	0 (critical safety metric)
Training accuracy (quantized)	67.3% (conservative bias intentional)
Scenario tests passed	19/22 (3 failures = extra cautious)
Total NN parameters	59 (fits in <1KB of logic)
Motor PWM speed	120/255 (47% duty, demo-safe)
Stuck recovery timeout	5 seconds without FORWARD
Power mode	Fully portable (dual LiPo, no cables)

9. Attached Code Files

All source code is provided in the accompanying **code/** folder:

DE1SoC_Robot_Report/

├── DE1SoC_Robot_Report.docx (this document)

└── code/

 ├── fpga/

 ├── robot_neural.v (top-level module)

 ├── neural_net.v (neural network + trained weights)

 ├── uart_rx.v (UART receiver)

 ├── uart_tx.v (UART transmitter)

 └── pin_assignments.tcl (FPGA pin mappings)

 ├── arduino/

 ├── de1soc_phase5_3sensor.ino (main control sketch)

 └── de1soc_data_collect_v2.ino (data collection sketch)

 └── training/

 └── fpga_nn_v5.ipynb (complete training pipeline)

End of Report